

FLEETLOGIX

DATA WAREHOUSE & PIPELINE ETL AUTOMATIZADO

Proyecto integrador: **AVANCE 3**

Alumno: DENNYS DURAN

Materia: **dspt06**

Objetivo

- Diseñar e implementar un Data Warehouse en Snowflake utilizando un modelo estrella.
 - Construir un pipeline ETL automatizado en Python para extraer datos desde PostgreSQL, transformarlos y cargarlos diariamente en Snowflake.
 - Habilitar análisis históricos y estratégicos sin afectar el sistema transaccional de la empresa.
-

Contexto del Proyecto

En los Avances 1 y 2 se construyó una base de datos operacional (OLTP) en PostgreSQL con 505,797 registros distribuidos en 6 tablas, además de 12 consultas SQL optimizadas mediante índices compuestos.

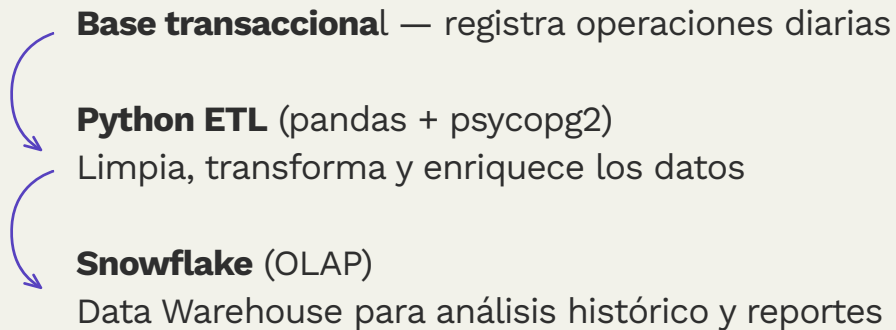
El Avance 3 representa la transición hacia el entorno analítico (OLAP): se implementa un Data Warehouse en Snowflake con modelo estrella y un pipeline ETL automatizado en Python.

Justificación de negocio: FleetLogix necesita responder preguntas estratégicas como:

- ¿Cuál fue la eficiencia promedio por ciudad en el cuarto trimestre?
- ¿Qué conductor generó más revenue en 2024?
- ¿Cómo evolucionó el desempeño operativo por turno y tipo de vehículo?

PostgreSQL está optimizado para transacciones del día a día, mientras que Snowflake permite ejecutar consultas analíticas sobre grandes volúmenes históricos sin afectar las operaciones.

ARQUITECTURA OLTP → ETL → OLAP



PREPARACIÓN DE POSTGRESQL

Columna `is_damaged` en `deliveries`

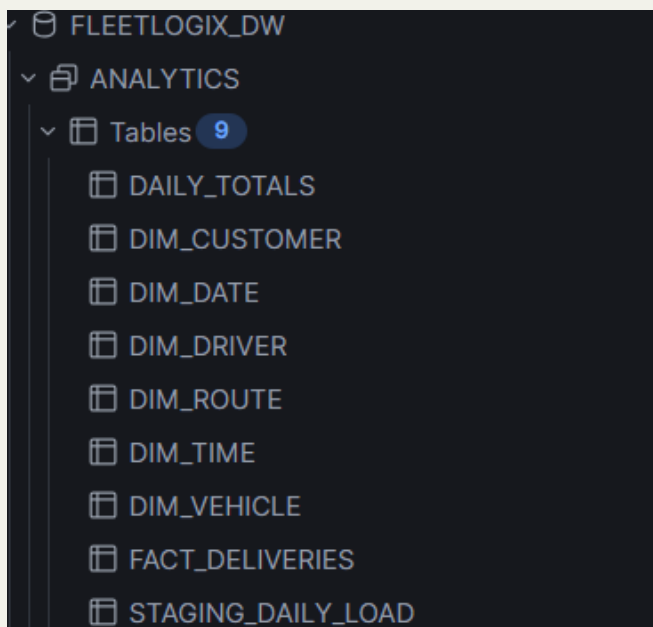
El pipeline requería un indicador de daño en los paquetes que no existía en la base operacional. Se agregó la columna `is_damaged` (BOOLEAN) a la tabla `deliveries`.

Decisión de diseño: el daño solo puede registrarse en entregas con estado `delivered`, ya que en la operación real FleetLogix retiene los paquetes dañados antes de enviarlos.

Se asignó un 3% de valores `TRUE` de forma aleatoria entre las entregas completadas, valor razonable para una empresa de logística de última milla.

Modelo Estrella en Snowflake

Se implementó un modelo estrella con una tabla central de hechos y seis dimensiones. Este diseño separa las métricas del contexto descriptivo y mejora el rendimiento de las consultas analíticas.



Tablas implementadas en el Data Warehouse de Snowflake

Se observa la estructura principal del Data Warehouse implementado en Snowflake. La solución está compuesta por una tabla de hechos (**FACT_DELIVERIES**) y seis dimensiones (**DIM_DATE**, **DIM_TIME**, **DIM_VEHICLE**, **DIM_DRIVER**, **DIM_ROUTE** y **DIM_CUSTOMER**), además de la tabla agregada **DAILY_TOTALS**, utilizada para optimizar consultas ejecutivas y reportes operativos.

TABLA	REGISTROS	FUENTE	DESCRIPCIÓN
dim_date	10	Python	Calendario del 22 al 31 de diciembre de 2024 con atributos temporales.
dim_time	24	Python	Horas del día clasificadas por turnos operativos.
dim_vehicle	200	PostgreSQL	Vehículos enriquecidos con antigüedad y fecha de mantenimiento.
dim_driver	400	PostgreSQL	Conductores con experiencia y categoría de desempeño.
dim_route	50	PostgreSQL	Rutas con nivel de dificultad y tipo de carretera.
dim_customer	10,000	PostgreSQL	Clientes más frecuentes clasificados por categoría.
fact_deliveries	320,238	ETL	Tabla central de hechos con métricas por entrega.
daily_totals	1	ETL	Resumen diario precalculado para reportes rápidos.

Justificación de daily_totals: se creó una tabla agregada para reducir tiempos de respuesta en consultas ejecutivas frecuentes, evitando recalcular métricas históricas sobre la tabla de hechos completa.

Decisiones de Diseño

1 dim_date: rango de 10 días

La base de datos es estática y no recibe datos nuevos. Un pipeline diario generaría únicamente un día en la dimensión temporal, por lo que se poblaron los últimos 10 días del rango disponible (22–31 dic 2024) para demostrar capacidades analíticas como comparaciones diarias y tendencias recientes.



Results (just now)

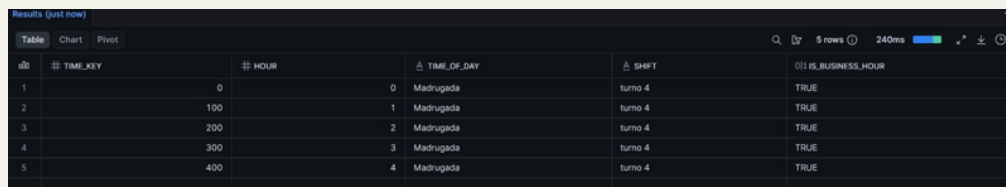
	# DATE_KEY	△ FULL_DATE	△ DAY_NAME	01 IS_WEEKEND	# QUARTER	# YEAR
1	20241222	2024-12-22	Sunday	TRUE	4	2024
2	20241223	2024-12-23	Monday	FALSE	4	2024
3	20241224	2024-12-24	Tuesday	FALSE	4	2024
4	20241225	2024-12-25	Wednesday	FALSE	4	2024
5	20241226	2024-12-26	Thursday	FALSE	4	2024

2 dim_time: 4 turnos operativos

Los turnos se definieron según la distribución horaria de la flota modelada en el Avance 1:

- **Mañana (06–11h):** operan todos los tipos de vehículos.
- **Tarde (12–19h):** pico de actividad de Vans y Motocicletas.
- **Noche (20–23h):** predominan los Camiones.
- **Madrugada (00–05h):** operación exclusiva de Camiones para rutas largas.

Esto permite analizar productividad y carga operativa por turno.



Results (just now)

	# TIME_KEY	# HOUR	△ TIME_OF_DAY	△ SHIFT	01 IS_BUSINESS_HOUR
1		0	0 Madrugada	turno 4	TRUE
2		100	1 Madrugada	turno 4	TRUE
3		200	2 Madrugada	turno 4	TRUE
4		300	3 Madrugada	turno 4	TRUE
5		400	4 Madrugada	turno 4	TRUE

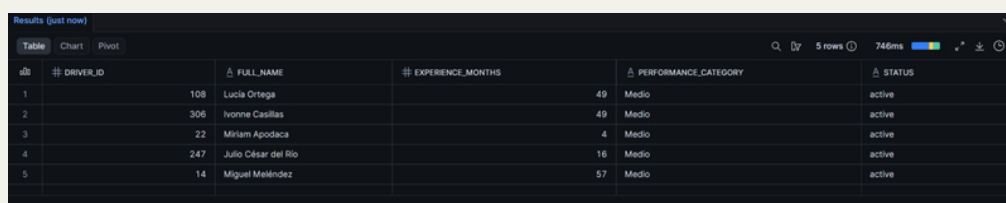
3 dim_driver: performance_category

La categoría de desempeño se calcula como tasa de entregas exitosas = delivered / total.

Los umbrales definidos fueron:

- **Alto: $\geq 85\%$**
- **Medio: $\geq 70\%$**
- **Bajo: $< 70\%$**

Justificación: el conductor no decide la ruta asignada; por ello, el desempeño se mide por la capacidad de completar entregas exitosamente y no por kilómetros recorridos.



Results (just now)

	# DRIVER_ID	△ FULL_NAME	# EXPERIENCE_MONTHS	△ PERFORMANCE_CATEGORY	△ STATUS
1	108	Lucía Ortega	49	Medio	active
2	306	Ivonne Casillas	49	Medio	active
3	22	Miriam Apodaca	4	Medio	active
4	247	Julio César del Río	16	Medio	active
5	14	Miguel Meléndez	57	Medio	active

4 dim_route: difficulty_level y route_type

difficulty_level se calcula según la distancia de la ruta:

Fácil: < 500 km

Medio: 500–1,500 km

Difícil: > 1,500 km

La clasificación cubre rutas reales del negocio, desde CDMX–Puebla (130 km) hasta Tijuana–CDMX (3,360 km).

route_type se deriva del código de ruta:

EXP: Autopista

FED: Federal

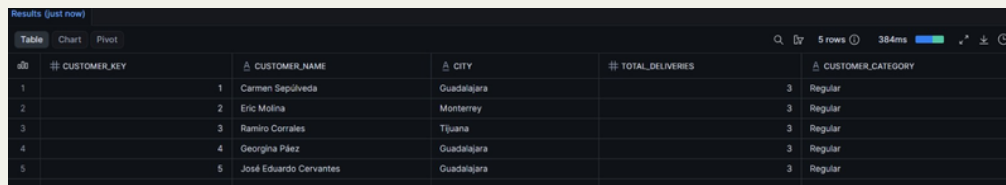
ECO: Libre

5 dim_customer: top 10,000 clientes

Faker generó nombres únicos para cada entrega, produciendo 383,473 clientes distintos. Cargar todos mediante operaciones MERGE individuales incrementaba significativamente el tiempo del ETL.

Se decidió conservar únicamente los 10,000 clientes con mayor frecuencia de entregas, ordenados por total_deliveries DESC.

Justificación de negocio: en logística, los clientes recurrentes representan la mayor parte del valor comercial y son los más relevantes para análisis de retención y revenue.



The screenshot shows a data table with the following columns: # CUSTOMER_KEY, CUSTOMER_NAME, CITY, TOTAL_DELIVERIES, and CUSTOMER_CATEGORY. The table contains 5 rows of data.

# CUSTOMER_KEY	CUSTOMER_NAME	CITY	TOTAL_DELIVERIES	CUSTOMER_CATEGORY
1	Carmen Sepúlveda	Guadalajara	3	Regular
2	Eric Molina	Monterrey	3	Regular
3	Ramiro Corrales	Tijuana	3	Regular
4	Georgina Pérez	Guadalajara	3	Regular
5	José Eduardo Cervantes	Guadalajara	3	Regular

6 customer_category: segmentación por frecuencia

- **Premium (≥ 10 entregas):** clientes de alto valor y relación consolidada.
- **Regular (3–9 entregas):** clientes activos con potencial de crecimiento.
- **Ocasional (< 3 entregas):** clientes de baja frecuencia o nuevos.

7 Time Travel: 20 días de retención

Snowflake permite consultar versiones históricas de las tablas. Se configuró una retención de 20 días para recuperar datos eliminados accidentalmente y facilitar auditorías operativas.

8 Control de acceso basado en roles (RBAC)

Se crearon vistas seguras específicas por área:

- **v_sales_deliveries:** acceso para Ventas, excluyendo clientes gubernamentales por restricciones contractuales.
- **v_operations_deliveries:** acceso completo para Operaciones con todos los indicadores operativos.

7. Pipeline ETL — Decisiones Técnicas

1 Inserción por lotes (batch_size = 1,000)

Snowflake tiene un límite de 200,000 expresiones por INSERT. Para cargar los 320,238 registros de fact_deliveries, se implementó inserción por lotes de 1,000 registros usando executemany, resultando en 321 lotes exitosos.

2 Uso de pg_cursor en lugar de pd.read_sql

La versión de pandas utilizada junto con Python 3.14 requería SQLAlchemy para conexiones con psycopg2 mediante pd.read_sql. Para simplificar dependencias y mejorar rendimiento, se optó por usar pg_cursor.execute() + fetchall() directamente.

3 Credenciales mediante .env

Las credenciales se almacenan en un archivo credenciales.env separado del código fuente, evitando exponer información sensible en el repositorio.

4 SCD Type 2 para dimensiones históricas

Las dimensiones dim_vehicle y dim_driver implementan Slowly Changing Dimensions Type 2, incorporando las columnas valid_from, valid_to e is_current.

Esto permite conservar el historial completo de cambios. Por ejemplo, si un conductor pasa de categoría Bajo a Alto, ambas versiones permanecen registradas con sus periodos de vigencia.

8. Retos Encontrados

- **Límite de inserciones en Snowflake:** obligó a implementar carga por lotes.
- **Compatibilidad pandas / psycopg2 / Python 3.14:** se resolvió evitando dependencias innecesarias de SQLAlchemy.
- **Volumen excesivo de clientes generados por Faker:** se mitigó mediante segmentación y priorización de clientes frecuentes.
- **Seguridad de credenciales:** se resolvió usando variables de entorno y archivos .env.

9. Métricas del Pipeline

MÉTRICA	VALOR
Registros extraídos de PostgreSQL	320,238
Registros transformados	320,238
Registros cargados en Snowflake	320,238
Ejecución exitosa sin errores	SÍ
Tiempo de ejecución	638 segundos (~10.6 min)
Lotes de fact_deliveries	321 lotes de 1,000 registros

10. Conclusión

El Avance 3 transforma la plataforma FleetLogix desde una arquitectura transaccional hacia una arquitectura analítica moderna.

Mediante la implementación de un Data Warehouse en Snowflake, un modelo estrella y un pipeline ETL automatizado, la organización puede:

- **realizar análisis históricos y comparativos,**
- **segmentar clientes y operaciones,**
- **monitorear KPIs logísticos,**
- **y generar reportes sin afectar el sistema operacional.**

La solución deja preparada la base para futuras capas de visualización en Power BI y para proyectos de analítica avanzada y machine learning.